Assignment-2

What to Hand In

You should upload a pdf or word file for **Problem-1** and **Problem-2**.

For the other problems, use the provided assignment link.

Problem-3 (7 points):

Write a **recursive** function that returns the number of even digits in an integer using the following header:

```
int evenCount(unsigned int value);
```

Write a test program that prompts the user to enter an integer, invokes the function and displays the number of even digits in it. Zero is an even digit.

Here are some sample runs.

```
Enter the number 4
The number of even digits is 1

Enter the number 1
The number of even digits is 0

Enter the number 0
The number of even digits is 1

Enter the number 10434052
The number of even digits is 5
```

Problem-4 (7 points):

Craps is a popular dice game played in casinos. Write a program to play a variation of the game, as follows:

Roll two dice. Each die has six faces representing values 1, 2, … , and 6, respectively. Check the sum of the two dice. If the sum is 2, 3, or 12 (called craps), you lose; if the sum is 7 or 11 (called natural), you win; if the sum is another value (i.e., 4, 5, 6, 8, 9, or 10), a point is established. Continue until you roll either a 7 (you lose) or the same point value (you win). Check the craps game rules online if something is unclear.

In the main function get the seed for the random numbers first and run the game with the scanned seed after that.

Here are some sample runs:

1.

```
Enter the seed 4
You rolled 6 + 6 = 12
You lose
```

2.

```
Enter the seed 1
You rolled 3 + 6 = 9
You rolled 5 + 6 = 11
You rolled 1 + 1 = 2
You rolled 2 + 6 = 8
You rolled 1 + 2 = 3
You rolled 1 + 3 = 4
You rolled 2 + 3 = 5
You rolled 3 + 5 = 8
You rolled 3 + 6 = 9
You win
```

3.

```
Enter the seed 2
You rolled 3 + 2 = 5
You rolled 1 + 6 = 7
You lose
```

For this problem you are given the starting code with random number generation.

How to Generate a Random Number Inside a Function in C++

1. **Pass the Random Number Engine as a Reference** Create a `std::mt19937` engine inside the main function. Pass it to the function by reference to avoid copying and ensure consistent state across calls.
2. **Form a Distribution with the Desired Range Inside the Function** Inside the function, define a distribution (e.g., `std::uniform_int_distribution` for integers to specify the range of random numbers.
3. **Generate Random Numbers** Use the distribution with the engine to produce a random number within the specified range.

```cpp
int f1(std::mt19937& eng) {
    std::uniform_int_distribution<int> dist(-10, 10);

    return dist(eng);
}

int main(void) {
    int seed;
    std::cout << "Enter the seed ";
    std::cin >> seed;

    // Initialize a random number engine with a seed
    std::mt19937 eng(seed);

    std::cout << f1(eng);

    return 0;
}
```

Problem-5 (7 points):

Create a function that joins an array of strings on a separator characters using the following header:

```
std::string my_join(const std::vector<std::string>& words, std::string
separ);
```

The function receives two parameters, a vector with all the strings and a separator.
The function returns a string where all the strings from vector are joined with the separator.

Write a test program that scans the words and adds them to the vector of strings. Use the word “exit” to stop scanning. The program also scans the separator, joins strings and prints the result.

Here are some sample runs.

1.

```
Enter strings ("exit" to stop)
what
is
your
name
exit
Enter the separator
^^^^
what^^^^is^^^^your^^^^name
```

2.

```
Enter strings ("exit" to stop)
hello
how
are
you
exit
Enter the separator
$
hello$how$are$you
```

Problem 5. Bonus (5 pts)

Too easy? Create another function **my_join_comb** instead. The function should print all the possible combinations of strings with separators. In the test program run my_join_comb with scanned words and a separator used in the previous problem.

Here are some sample runs:

```
std::vector<std::string> v = { "I", "love", "coding" };
my_join_comb(v, ".");
```

```
I.love.coding
I.coding.love
love.I.coding
love.coding.I
coding.I.love
coding.love.I
```

```
std::vector<std::string> v = { "brown", "town", "clown", "crown" };
my_join_comb(v, "--");
```

```
brown--town--clown--crown
brown--town--crown--clown
brown--clown--town--crown
brown--clown--crown--town
brown--crown--town--clown
brown--crown--clown--town
town--brown--clown--crown
town--brown--crown--clown
town--clown--brown--crown
town--clown--crown--brown
town--crown--brown--clown
town--crown--clown--brown
clown--brown--town--crown
clown--brown--crown--town
clown--town--brown--crown
clown--town--crown--brown
clown--crown--brown--town
clown--crown--town--brown
crown--brown--town--clown
crown--brown--clown--town
crown--town--brown--clown
crown--town--clown--brown
crown--clown--brown--town
crown--clown--town--brown
```